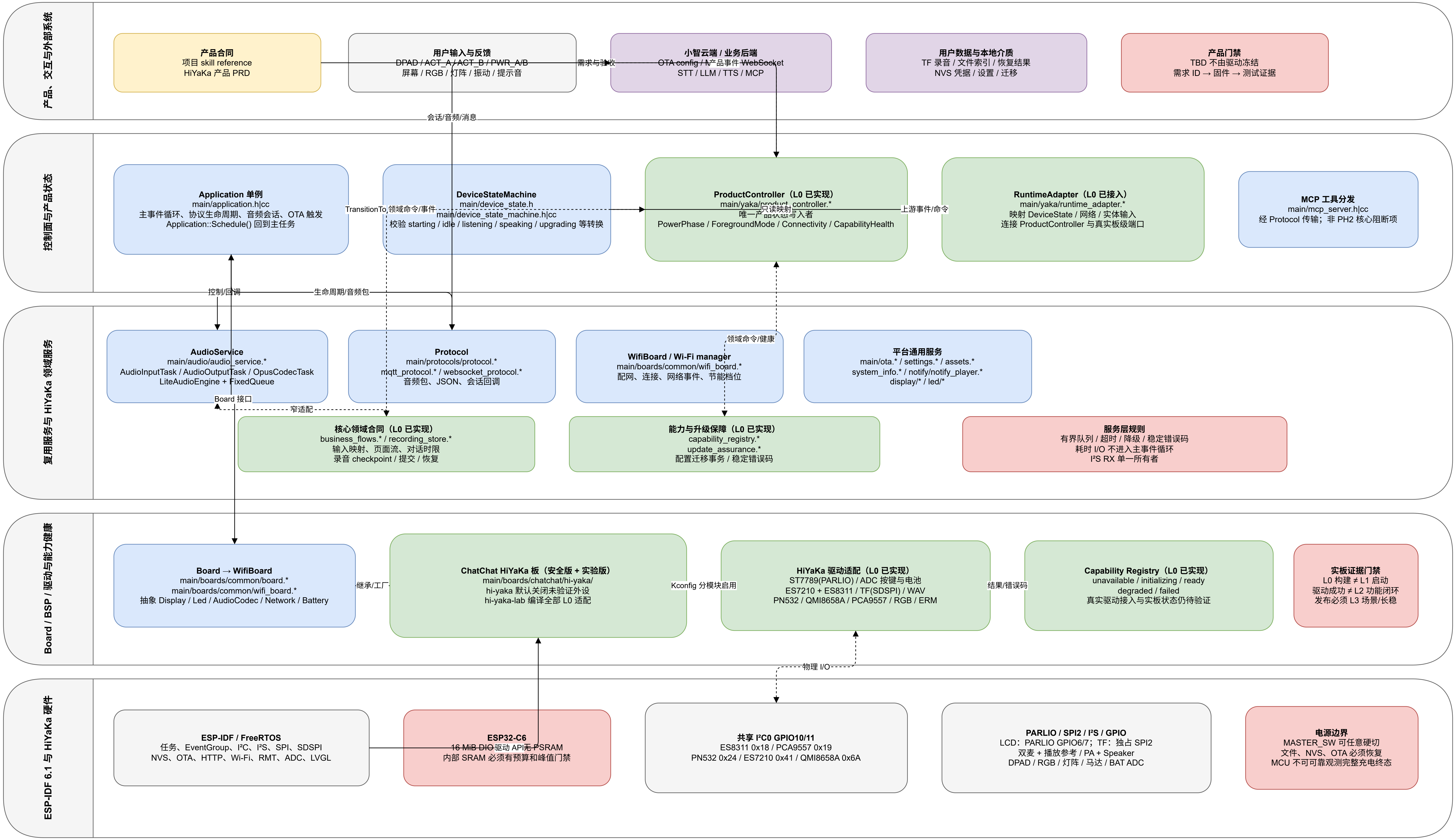
HiYaKa 固件总体架构

小智 v2.5.2 / ESP-IDF 6.1 + M2–M6 软件 L0 适配



产品、交互与外部系统

产品合同
项目 skill reference
HiYaKa 产品 PRD

用户输入与反馈
DPAD / ACT_A / ACT_B / PWR_A/B
屏幕 / RGB / 灯阵 / 振动 / 提示音

小智云端 / 业务后端
OTA config / 产品事件 WebSocket
STT / LLM / TTS / MCP

用户数据与本地介质
TF 录音 / 文件索引 / 恢复结果
NVS 凭据 / 设置 / 迁移

产品门禁
TBD 不由驱动冻结
需求 ID → 固件 → 测试证据

控制面与产品状态

Application 单例
main/application.h|cc
主事件循环、协议生命周期、音频会话、OTA 触发
Application::Schedule() 回到主任务

DeviceStateMachine
main/device_state.h
main/device_state_machine.h|cc
校验 starting / idle / listening / speaking / upgrading 等转换

ProductController（L0 已实现）
main/yaka/product_controller.*
唯一产品状态写入者
PowerPhase / ForegroundMode / Connectivity / CapabilityHealth

RuntimeAdapter（L0 已接入）
main/yaka/runtime_adapter.*
映射 DeviceState / 网络 / 实体输入
连接 ProductController 与真实板级端口

MCP 工具分发
main/mcp_server.h|cc
经 Protocol 传输；非 PH2 核心阻断项

复用服务与 HiYaKa 领域服务

AudioService
main/audio/audio_service.*
AudioInputTask / AudioOutputTask / OpusCodecTask
LiteAudioEngine + FixedQueue

Protocol
main/protocols/protocol.*
mqtt_protocol.* / websocket_protocol.*
音频包、JSON、会话回调

WifiBoard / Wi-Fi manager
main/boards/common/wifi_board.*
配网、连接、网络事件、节能档位

平台通用服务
main/ota.* / settings.* / assets.*
system_info.* / notify/notify_player.*
display/* / led/*

核心领域合同（L0 已实现）
business_flows.* / recording_store.*
输入映射、页面流、对话时限
录音 checkpoint / 提交 / 恢复

能力与升级保障（L0 已实现）
capability_registry.*
update_assurance.*
配置迁移事务 / 稳定错误码

服务层规则
有界队列 / 超时 / 降级 / 稳定错误码
耗时 I/O 不进入主事件循环
I²S RX 单一所有者

Board / BSP / 驱动与能力健康

Board → WifiBoard
main/boards/common/board.*
main/boards/common/wifi_board.*
抽象 Display / Led / AudioCodec / Network / Battery

ChatChat HiYaKa 板（安全版 + 实验版）
main/boards/chatchat/hi-yaka/
hi-yaka 默认关闭未验证外设
hi-yaka-lab 编译全部 L0 适配

HiYaKa 驱动适配（L0 已实现）
ST7789(PARLIO) / ADC 按键与电池
ES7210 + ES8311 / TF(SDSPI) / WAV
PN532 / QMI8658A / PCA9557 / RGB / ERM

Capability Registry（L0 已实现）
unavailable / initializing / ready
degraded / failed
真实驱动接入与实板状态仍待验证

实板证据门禁
L0 构建 ≠ L1 启动
驱动成功 ≠ L2 功能闭环
发布必须 L3 场景/长稳

ESP-IDF 6.1 与 HiYaKa 硬件

ESP-IDF / FreeRTOS
任务、EventGroup、I²C、I²S、SPI、SDSPI
NVS、OTA、HTTP、Wi-Fi、RMT、ADC、LVGL

ESP32-C6
16 MiB DIO 无 PSRAM
内部 SRAM 必须有预算和峰值门禁

共享 I²C0 GPIO10/11
ES8311 0x18 / PCA9557 0x19
PN532 0x24 / ES7210 0x41 / QMI8658A 0x6A

PARLIO / SPI2 / I²S / GPIO
LCD：PARLIO GPIO6/7；TF：独占 SPI2
双麦 + 播放参考 / PA + Speaker
DPAD / RGB / 灯阵 / 马达 / BAT ADC

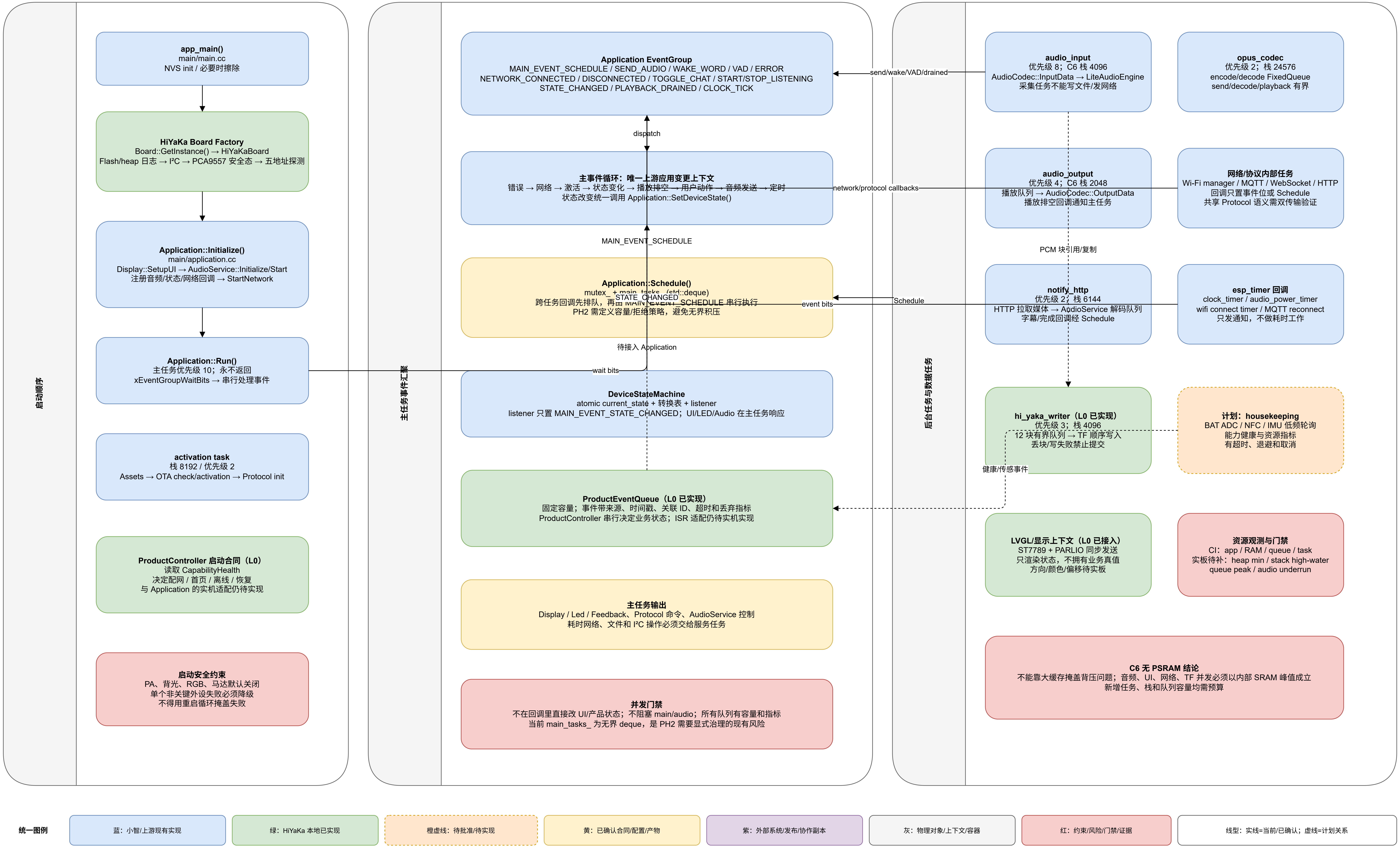
电源边界
MASTER_SW 可任意硬切
文件、NVS、OTA 必须恢复
MCU 不可可靠观测完整充电终态

统一图例

- 蓝：小智/上游现有实现
- 绿：HiYaKa 本地已实现
- 橙虚线：待批准/待实现
- 黄：已确认合同/配置/产物
- 紫：外部系统/发布/协作副本
- 灰：物理对象/上下文/容器
- 红：约束/风险/门禁/证据
- 线型：实线=当前/已确认；虚线=计划关系

运行时控制面、事件汇聚与 FreeRTOS 任务

实线为当前/已确认关系；虚线为计划关系



启动顺序

app_main()
main/main.cc
NVS init / 必要时擦除

HiYaKa Board Factory
Board::GetInstance() → HiYaKaBoard
Flash/heap 日志 → I²C → PCA9557 安全态 → 五地址探测

Application::Initialize()
main/application.cc
Display::SetupUI → AudioService::Initialize/Start
注册音频/状态/网络回调 → StartNetwork

Application::Run()
主任务优先级 10；永不返回
xEventGroupWaitBits → 串行处理事件

activation task
栈 8192 / 优先级 2
Assets → OTA check/activation → Protocol init

ProductController 启动合同（L0）
读取 CapabilityHealth
决定配网 / 首页 / 离线 / 恢复
与 Application 的实机适配仍待实现

启动安全约束
PA、背光、RGB、马达默认关闭
单个非关键外设失败必须降级
不得用重启循环掩盖失败

主任务事件汇聚

Application EventGroup
MAIN_EVENT_SCHEDULE / SEND_AUDIO / WAKE_WORD / VAD / ERROR
NETWORK_CONNECTED / DISCONNECTED / TOGGLE_CHAT / START/STOP_LISTENING
STATE_CHANGED / PLAYBACK_DRAINED / CLOCK_TICK

主事件循环：唯一上游应用变更上下文
错误 → 网络 → 激活 → 状态变化 → 播放排空 → 用户动作 → 音频发送 → 定时
状态改变统一调用 Application::SetDeviceState()

Application::Schedule()
mutex_ + main_tasks_（std::deque)
跨任务回调先排队，再由 MAIN_EVENT_SCHEDULE 串行执行
PH2 需定义容量/拒绝策略，避免无界积压

DeviceStateMachine
atomic current_state + 转换表 + listener
listener 只置 MAIN_EVENT_STATE_CHANGED；UI/LED/Audio 在主任务响应

ProductEventQueue（L0 已实现）
固定容量；事件带来源、时间戳、关联 ID、超时和丢弃指标
ProductController 串行决定业务状态；ISR 适配仍待实机实现

主任务输出
Display / Led / Feedback、Protocol 命令、AudioService 控制
耗时网络、文件和 I²C 操作必须交给服务任务

并发门禁
不在回调里直接改 UI/产品状态；不阻塞 main/audio；所有队列有容量和指标
当前 main_tasks_ 为无界 deque，是 PH2 需要显式治理的现有风险

后台任务与数据任务

audio_input
优先级 8；C6 栈 4096
AudioCodec::InputData → LiteAudioEngine
采集任务不能写文件/发网络

opus_codec
优先级 2；栈 24576
encode/decode FixedQueue
send/decode/playback 有界

audio_output
优先级 4；C6 栈 2048
播放队列 → AudioCodec::OutputData
播放排空回调通知主任务

网络/协议内部任务
Wi-Fi manager / MQTT / WebSocket / HTTP
回调只置事件位或 Schedule
共享 Protocol 语义需双传输验证

notify_http
优先级 2；栈 6144
HTTP 拉取媒体 → AudioService 解码队列
字幕/完成回调经 Schedule

esp_timer 回调
clock_timer / audio_power_timer
wifi connect timer / MQTT reconnect
只发通知，不做耗时工作

hi_yaka_writer（L0 已实现）
优先级 3；栈 4096
12 块有界队列 → TF 顺序写入
丢块/写失败禁止提交

计划：housekeeping
BAT ADC / NFC / IMU 低频轮询
能力健康与资源指标
有超时、退避和取消

LVGL/显示上下文（L0 已接入）
ST7789 + PARLIO 同步发送
只渲染状态，不拥有业务真值
方向/颜色/偏移待实板

资源观测与门禁
CI：app / RAM / queue / task
实板待补：heap min / stack high-water
queue peak / audio underrun

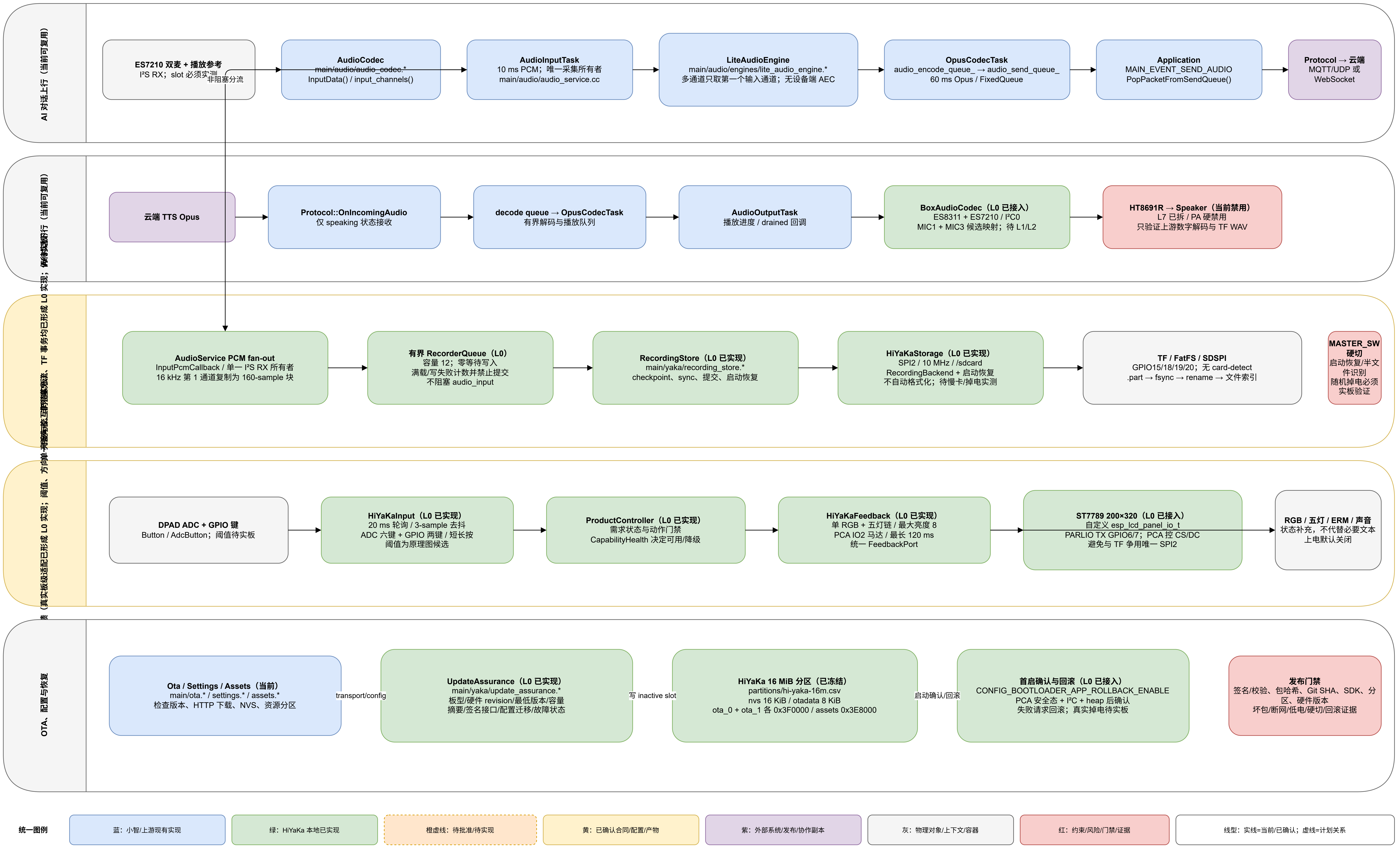
C6 无 PSRAM 结论
不能靠大缓存掩盖背压问题；音频、UI、网络、TF 并发必须以内部 SRAM 峰值成立
新增任务、栈和队列容量均需预算

统一图例

🟦	蓝：小智/上游现有实现
🟩	绿：HiYaKa 本地已实现
🟧	橙虚线：待批准/待实现
🟨	黄：已确认合同/配置/产物
🟪	紫：外部系统/发布/协作副本
⬜	灰：物理对象/上下文/容器
🟥	红：约束/风险/门禁/证据
⬜	线型：实线=当前/已确认；虚线=计划关系

关键数据路径与所有权

展示当前上游链路、HiYaKa L0 合同与仍待实机接入的路径



AI 对话上行（当前可复用）

ES7210 双麦 + 播放参考
I^{2}S RX；slot 必须实测

非阻塞分流

AudioCodec
main/audio/audio_codec.*
InputData() / input_channels()

AudioInputTask
10 ms PCM；唯一采集所有者
main/audio/audio_service.cc

LiteAudioEngine
main/audio/engines/lite_audio_engine.*
多通道只取第一个输入通道；无设备端 AEC

OpusCodecTask
audio_encode_queue_ → audio_send_queue_
60 ms Opus / FixedQueue

Application
MAIN_EVENT_SEND_AUDIO
PopPacketFromSendQueue()

Protocol → 云端
MQTT/UDP 或
WebSocket

[illegible]行（当前可复用）

云端 TTS Opus

Protocol::OnIncomingAudio
仅 speaking 状态接收

decode queue → OpusCodecTask
有界解码与播放队列

AudioOutputTask
播放进度 / drained 回调

BoxAudioCodec（L0 已接入）
ES8311 + ES7210 / I^{2}C0
MIC1 + MIC3 候选映射；待 L1/L2

HT8691R → Speaker（当前禁用）
L7 已拆 / PA 硬禁用
只验证上游数字解码与 TF WAV

[illegible]实现；[illegible]、TF 事务均已形成 L0 实现；[illegible]

AudioService PCM fan-out
InputPcmCallback / 单一 I^{2}S RX 所有者
16 kHz 第 1 通道复制为 160-sample 块

有界 RecorderQueue（L0）
容量 12；零等待写入
满载/写失败计数并禁止提交
不阻塞 audio_input

RecordingStore（L0 已实现）
main/yaka/recording_store.*
checkpoint、sync、提交、启动恢复

HiYaKaStorage（L0 已实现）
SPI2 / 10 MHz / /sdcard
RecordingBackend + 启动恢复
不自动格式化；待慢卡/掉电实测

TF / FatFS / SDSPI
GPIO15/18/19/20；无 card-detect
.part → fsync → rename → 文件索引

MASTER_SW 硬切
启动恢复/半文件识别
随机掉电必须实板验证

[illegible]（真实板级适配已形成 L0 实现；阈值、方向[illegible]

DPAD ADC + GPIO 键
Button / AdcButton；阈值待实板

HiYaKaInput（L0 已实现）
20 ms 轮询 / 3-sample 去抖
ADC 六键 + GPIO 两键 / 短长按
阈值为原理图候选

ProductController（L0 已实现）
需求状态与动作门禁
CapabilityHealth 决定可用/降级

HiYaKaFeedback（L0 已实现）
单 RGB + 五灯链 / 最大亮度 8
PCA IO2 马达 / 最长 120 ms
统一 FeedbackPort

ST7789 200×320（L0 已接入）
自定义 esp_lcd_panel_io_t
PARLIO TX GPIO6/7；PCA 控 CS/DC
避免与 TF 争用唯一 SPI2

RGB / 五灯 / ERM / 声音
状态补充，不代替必要文本
上电默认关闭

OTA、配置与恢复

Ota / Settings / Assets（当前）
main/ota.* / settings.* / assets.*
检查版本、HTTP 下载、NVS、资源分区

transport/config

UpdateAssurance（L0 已实现）
main/yaka/update_assurance.*
板型/硬件 revision/最低版本/容量
摘要/签名接口/配置迁移/故障状态

写 inactive slot

HiYaKa 16 MiB 分区（已冻结）
partitions/hi-yaka-16m.csv
nvs 16 KiB / otadata 8 KiB
ota_0 + ota_1 各 0x3F0000 / assets 0x3E8000

启动确认/回滚

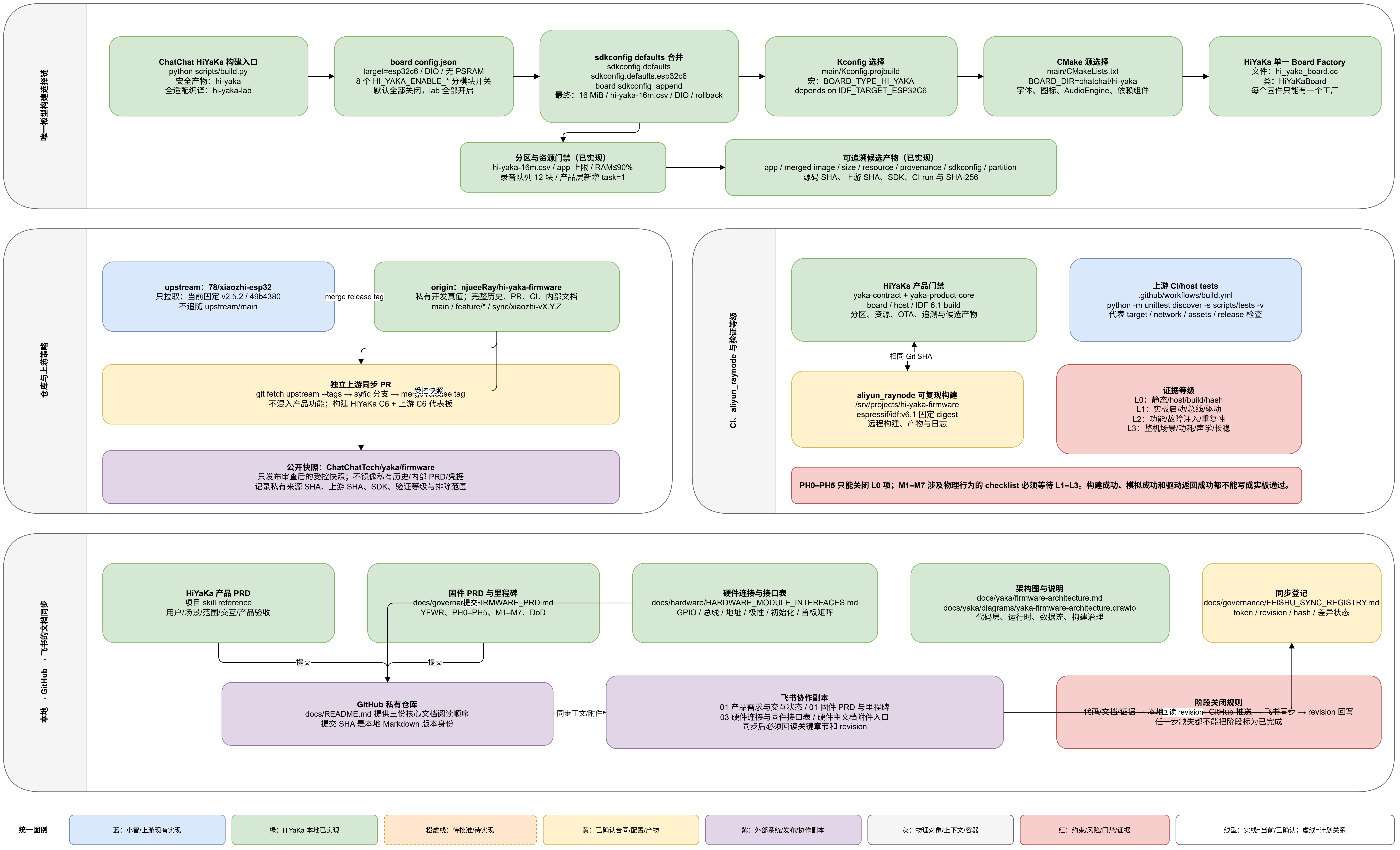
首启确认与回滚（L0 已接入）
CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE
PCA 安全态 + I^{2}C + heap 后确认
失败请求回滚；真实掉电待实板

发布门禁
签名/校验、包哈希、Git SHA、SDK、分区、硬件版本
坏包/断网/低电/硬切/回滚证据

统一图例

🟦	蓝：小智/上游现有实现
🟩	绿：HiYaKa 本地已实现
🟧	橙虚线：待批准/待实现
🟨	黄：已确认合同/配置/产物
🟪	紫：外部系统/发布/协作副本
⬜	灰：物理对象/上下文/容器
🟥	红：约束/风险/门禁/证据
	线型：实线=当前/已确认；虚线=计划关系

构建选择链、上游同步、CI 与三端文档治理



唯一板型构建选择链

ChatChat HiYaKa 构建入口
python scripts/build.py
安全产物：hi-yaka
全适配编译：hi-yaka-lab

board config.json
target=esp32c6 / DIO / 无 PSRAM
8 个 HI_YAKA_ENABLE_* 分模块开关
默认全部关闭，lab 全部开启

sdkconfig defaults 合并
sdkconfig.defaults
sdkconfig.defaults.esp32c6
board sdkconfig_append
最终：16 MiB / hi-yaka-16m.csv / DIO / rollback

Kconfig 选择
main/Kconfig.projbuild
宏：BOARD_TYPE_HI_YAKA
depends on IDF_TARGET_ESP32C6

CMake 源选择
main/CMakeLists.txt
BOARD_DIR=chatchat/hi-yaka
字体、图标、AudioEngine、依赖组件

HiYaKa 单一 Board Factory
文件：hi_yaka_board.cc
类：HiYaKaBoard
每个固件只能有一个工厂

分区与资源门禁（已实现）
hi-yaka-16m.csv / app 上限 / RAM≤90%
录音队列 12 块 / 产品层新增 task=1

可追溯候选产物（已实现）
app / merged image / size / resource / provenance / sdkconfig / partition
源码 SHA、上游 SHA、SDK、CI run 与 SHA-256

仓库与上游策略

upstream：78/xiaozhi-esp32
只拉取；当前固定 v2.5.2 / 49b4380
不追随 upstream/main

merge release tag

origin：njueeRay/hi-yaka-firmware
私有开发真值；完整历史、PR、CI、内部文档
main / feature/* / sync/xiaozhi-vX.Y.Z

独立上游同步 PR
git fetch upstream --tags → sync 分支 → merge release tag
不混入产品功能；构建 HiYaKa C6 + 上游 C6 代表板

受控快照

公开快照：ChatChatTech/yaka/firmware
只发布审查后的受控快照；不镜像私有历史/内部 PRD/凭据
记录私有来源 SHA、上游 SHA、SDK、验证等级与排除范围

CI、aliyun_raynode 与验证等级

HiYaKa 产品门禁
yaka-contract + yaka-product-core
board / host / IDF 6.1 build
分区、资源、OTA、追溯与候选产物

上游 CI/host tests
.github/workflows/build.yml
python -m unittest discover -s scripts/tests -v
代表 target / network / assets / release 检查

相同 Git SHA

aliyun_raynode 可复现构建
/srv/projects/hi-yaka-firmware
espressif/idf:v6.1 固定 digest
远程构建、产物与日志

证据等级
L0：静态/host/build/hash
L1：实板启动/总线/驱动
L2：功能/故障注入/重复性
L3：整机场景/功耗/声学/长稳

PH0–PH5 只能关闭 L0 项；M1–M7 涉及物理行为的 checklist 必须等待 L1–L3。构建成功、模拟成功和驱动返回成功都不能写成实板通过。

本地 → GitHub → 飞书的文档同步

HiYaKa 产品 PRD
项目 skill reference
用户/场景/范围/交互/产品验收

固件 PRD 与里程碑
docs/governar提交IRMWARE_PRD.md
YFWR、PH0–PH5、M1–M7、DoD

硬件连接与接口表
docs/hardware/HARDWARE_MODULE_INTERFACES.md
GPIO / 总线 / 地址 / 极性 / 初始化 / 首板矩阵

架构图与说明
docs/yaka/firmware-architecture.md
docs/yaka/diagrams/yaka-firmware-architecture.drawio
代码层、运行时、数据流、构建治理

同步登记
docs/governance/FEISHU_SYNC_REGISTRY.md
token / revision / hash / 差异状态

提交

提交

GitHub 私有仓库
docs/README.md 提供三份核心文档阅读顺序
提交 SHA 是本地 Markdown 版本身份

同步正文/附件

飞书协作副本
01 产品需求与交互状态 / 01 固件 PRD 与里程碑
03 硬件连接与固件接口表 / 硬件主文档附件入口
同步后必须回读关键章节和 revision

阶段关闭规则
代码/文档/证据 → 本地回读 revision→ GitHub 推送 → 飞书同步 → revision 回写
任一步缺失都不能把阶段标为已完成

统一图例

- 蓝：小智/上游现有实现
- 绿：HiYaKa 本地已实现
- 橙虚线：待批准/待实现
- 黄：已确认合同/配置/产物
- 紫：外部系统/发布/协作副本
- 灰：物理对象/上下文/容器
- 红：约束/风险/门禁/证据
- 线型：实线=当前/已确认；虚线=计划关系